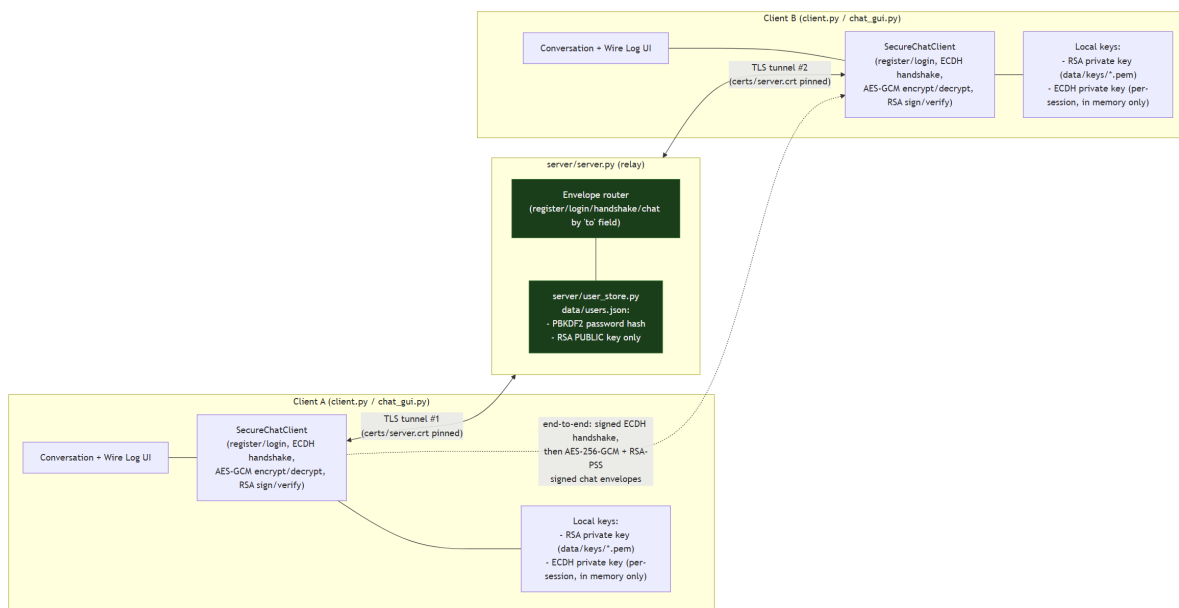


Secure Chat Application — Architecture

Architecture

Overview

Three processes talk over TCP: two `client/client.py` (or `gui/chat_gui.py`) instances and one `server/server.py` relay. Every connection is wrapped in TLS (Phase 6) end to end between each client and the server — but the server is a **relay, not a trusted third party for message content**: it routes envelopes by username and stores passwords (hashed) and public keys, but the AES-GCM session key and RSA private keys never reach it. The diagram below shows both boundaries at once: the outer TLS tunnels (what protects the pipe) and the inner end-to-end layer (what protects the content, even from the server itself).



Architecture diagram: Client A and Client B each run `SecureChatClient` with local RSA and per-session ECDH keys, connected to `server/server.py`'s relay over two separately TLS-pinned tunnels; the relay's envelope router and user store (dark green) only ever see routing metadata, password hashes, and RSA public keys, while a direct dotted line shows the end-to-end signed-ECDH-handshake-then-AES-GCM-and-RSA-PSS layer running between the two clients, invisible to the server.

A high-resolution version is also at [architecture_diagram.png](#) (2352px wide) and as a zoomable vector at [architecture_diagram.svg](#).

Mermaid source (for future edits — re-render with `npx @mermaid-js/mermaid-cli`)

```
graph LR
    subgraph ClientA ["Client A (client.py / chat_gui.py)"]
        A_UI["Conversation + Wire Log UI"]
        A_Logic["SecureChatClient\n(register/login, ECDH handshake,\nAES-GCM encrypt/decrypt, RSA sign/verify)"]
        A_Keys["Local keys:\n- RSA private key (data/keys/*.pem)\n- ECDH private key (per-session, in memory only)"]
        A_UI --- A_Logic
        A_Logic --- A_Keys
    end

    subgraph ClientB ["Client B (client.py / chat_gui.py)"]
        B_UI["Conversation + Wire Log UI"]
        B_Logic["SecureChatClient\n(register/login, ECDH handshake,\nAES-GCM encrypt/decrypt, RSA sign/verify)"]
        B_Keys["Local keys:\n- RSA private key (data/keys/*.pem)\n- ECDH private key (per-session, in memory only)"]
        B_UI --- B_Logic
        B_Logic --- B_Keys
    end

    subgraph Server ["server/server.py (relay)"]
        ER["Envelope router\n(register/login/handshake/chat by to field)"]
        US["server/user_store.py\ndata/users.json:\n- PBKDF2 password hash\n- RSA PUBLIC key only"]
    end

    ClientA -- "TLS tunnel #1 (certs/server.crt pinned)" --> Server
    ClientB -- "TLS tunnel #2 (certs/server.crt pinned)" --> Server
    ClientA -.-> ClientB
```

```

only)"]
    B_UI --- B_Logic
    B_Logic --- B_Keys
end

subgraph Server["server/server.py (relay)"]
    S_Route["Envelope router\n(register/login/handshake/chat by 'to' field)"]
    S_Store["server/user_store.py\data/users.json:\n- PBKDF2 password hash\n- RSA PUBLIC key only"]
    S_Route --- S_Store
end

A_Logic <-->|"TLS tunnel #1\n(certs/server.crt pinned)"| Server
Server <-->|"TLS tunnel #2\n(certs/server.crt pinned)"| B_Logic

A_Logic -.->|"end-to-end: signed ECDH handshake,\nthen AES-256-GCM + RSA-PSS\nsigned chat envelopes"| B_Logic

classDef canSee fill:#1a3d1a,color:#e8e8e8,stroke:#2f6b2f;
classDef cannotSee fill:#3d1a1a,color:#e8e8e8,stroke:#6b2f2f;
class S_Route,S_Store canSee;

```

What the server CAN see

- Usernames and the routing metadata of every envelope (who is talking to whom, and when — it decides delivery by the `to` field).
- Each user's PBKDF2 password *hash* (never the plaintext password — see `auth/password_hash.py`) and RSA *public* key (`server/user_store.py`), both submitted once at registration.
- Opaque ECDH public keys during the handshake, and opaque AES-GCM ciphertext/tag for every chat message — bytes it can relay but not read, since it never holds the session key.
- The plaintext of the register/login envelope *as JSON, before TLS encrypts it for transit* — i.e. the server, as the TLS endpoint, necessarily decrypts the connection to read routing envelopes. This is why the AES-GCM/RSA layer for chat *content* matters even though TLS is also present: TLS protects data in transit to the server, not from the server itself. A password is the one piece of data that has no additional application-layer protection beyond TLS — see “Known limitations” in the main [README.md](#).

What the server CANNOT see (by design, verified in `tests/`)

- The plaintext of any chat message, ever — only AES-256-GCM ciphertext (`tests/test_aes_gcm.py`, `tests/test_tls_setup.py`).
- Either side's ECDH private key, or the derived AES-256 session key (`crypto_engine/dh_exchange.py`; the private key is deleted from memory immediately after use, never serialized, never sent).
- Either side's RSA private key — only the public key, submitted once at registration (`auth/keystore.py`, `crypto_engine/signatures.py`).
- A user's plaintext password — only its PBKDF2 hash is ever persisted (`auth/password_hash.py`; `tests/test_password_hash.py::test_hash_does_not_contain_plaintext_password`).

Threat model this maps to

Attacker position	What stops them	Where
On the network, between a client and the server	TLS (Phase 6) — connection is opaque	<code>certs/</code> , <code>connect_tls()</code> in <code>client/client.py</code>
Controls/compromises the relay server itself	AES-256-GCM (content), RSA-PSS (handshake + message authenticity) — the server never holds the keys needed to read or forge either	<code>crypto_engine/aes_gcm.py</code> , <code>crypto_engine/signatures.py</code>
Captures a valid envelope and resends it later	Signed timestamp + freshness window + per-sender nonce tracking (Phase 7b)	<code>client/client.py</code> 's <code>is_timestamp_fresh</code> , <code>_seen_nonces</code>

Attacker position	What stops them	Where
Tries to brute-force a captured password hash	PBKDF2, 200,000+ iterations, per-user random salt	<code>auth/password_hash.py</code>

See the main [README.md](#) for the full wire protocol, the live demo scripts that exercise each of these rows against the real code, and the honest list of what's *not* covered by this threat model.